



2. Threads (continuação)

Celso Paím

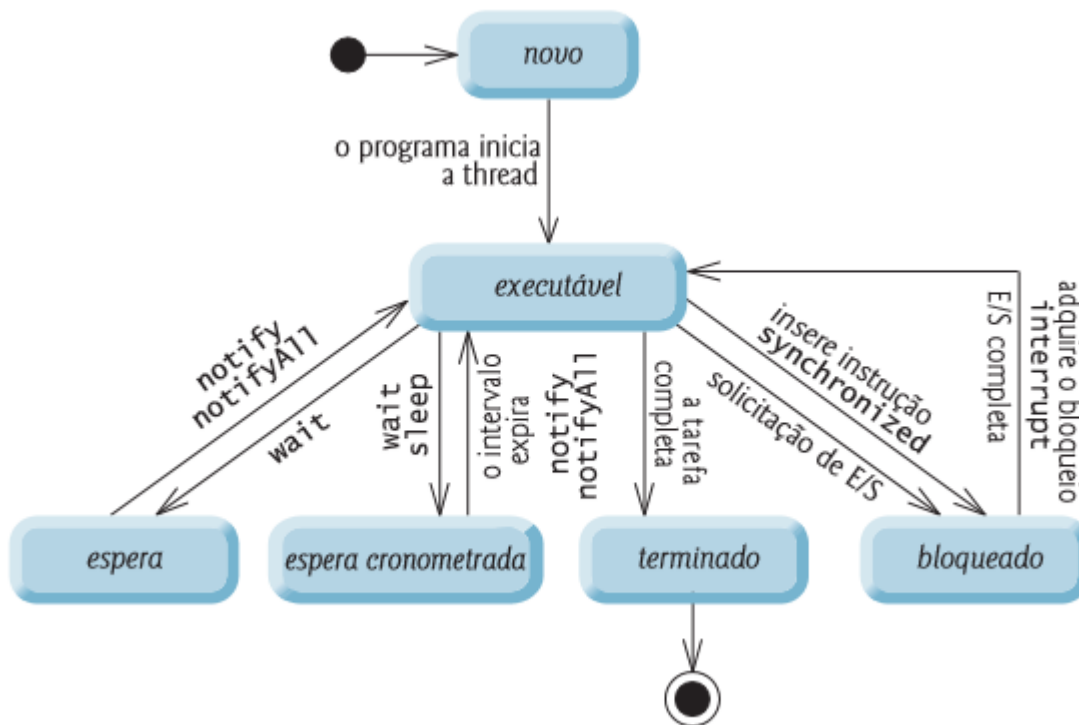


Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.2.1 Ciclo de vida (estados) de uma Thread





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.2.1 Ciclo de vida (estados) de uma Thread

NEW (Novo) – a *thread* é criada e permanece nesse estado até que o programa a inicie, invocando o método `start()`.

RUNNABLE (Executável) – a *thread* pronta para ser executada e/ou em execução (em processamento).

BLOCKED (Bloqueado) – uma *thread* executável passa para o estado bloqueado quando tenta realizar uma tarefa que não pode ser concluída imediatamente e deve esperar temporariamente até que a tarefa seja concluída.

WAITING (Espera) – por vezes a *thread* em estado RUNNABLE transita para o estado de espera enquanto aguarda outra *thread* realizar uma tarefa. Uma *thread* em espera volta ao estado executável somente quando outra *thread* a notifica para continuar a execução (através da invocação dos métodos `notify()` ou `notifyAll()`).



2.2.1 Ciclo de vida (estados) de uma Thread

TIMED_WAITING (Espera temporizada) - Uma *thread* executável pode entrar no estado de espera sincronizada por um intervalo especificado de tempo. Ela faz a transição de volta ao estado executável quando esse intervalo de tempo expira ou o evento que ela espera ocorre. *Threads* de espera sincronizada não podem utilizar um processador, mesmo se houver um disponível. Uma *thread* executável pode transitar para o estado de espera sincronizada se fornecer um intervalo de espera opcional quando ela estiver esperando outra *thread* realizar uma tarefa. Essa *thread* retornará ao estado executável quando ela for notificada por outra thread ou quando o intervalo sincronizado expirar — o que ocorrer primeiro.

TERMINATED (Terminada) - Uma *thread* executável entra no estado terminado, às vezes chamado estado morto, quando ela conclui com sucesso suas tarefas ou é de outro modo terminada (inclusive em razão de um erro).



2.2.1 Ciclo de vida (estados) de uma Thread

Há ainda o estado **READY (pronto)** que é um conceito visível ao Sistema Operativo mas não à JVM!

Na maioria dos sistemas operativos, cada *thread* recebe uma pequena quantidade de tempo de processador — chamada de *quantum* ou fração de tempo — com a qual realiza sua tarefa.

Quando o quantum expira, a *thread* retorna ao estado pronto, e o sistema operativo atribui outra *thread* ao processador. As transições entre os estados pronto e em execução são tratadas exclusivamente pelo sistema operativo. A JVM não “vê” as transições — ela simplesmente vê a *thread* como executável e deixa para o sistema operativo fazer a transição entre a *thread* pronta e em execução.

O processo que um sistema operativo usa para determinar qual *thread* despachar é chamado agendador de *thread* (*scheduler*, algumas vezes também confundido com *dispatcher*) e depende das prioridades de cada *thread*.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Jan/2026

// Demonstrar o uso de multiplas threads em tempos diferentes

```
public class ThreadTester {
```

```
    public static void main( String [] args ) {
```

```
        // criar e nomear as threads
```

```
        PrintThread thread1 = new PrintThread( "thread_APLHA" );
```

```
        PrintThread thread2 = new PrintThread( "thread_BETA" );
```

```
        PrintThread thread3 = new PrintThread( "thread_GAMA" );
```

```
        System.out.println( "Iniciando as threads" );
```

```
        thread1.start();
```

```
        thread2.start();
```

```
        thread3.start();
```

```
        System.out.println( "Inciadas as threads, termino das instrucoes do main" );
```

```
    }
```

```
} // fim da classe ThreadTester
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// classe PrintThread

```
class PrintThread extends Thread {  
    private int sleepTime;  
  
    public PrintThread( String name ) {  
        super( name );  
        sleepTime = ( int ) ( Math.random() * 5001 );  
    }  
  
    public void run() {  
        try {  
            System.out.println( getName() + " vai adormecer " + sleepTime + " ms" );  
            Thread.sleep( sleepTime );  
        }  
        catch ( InterruptedException exception ) {  
            exception.printStackTrace();  
        }  
        System.out.println( getName() + " acordou..." );  
    }  
}
```



Universidade Católica de Angola

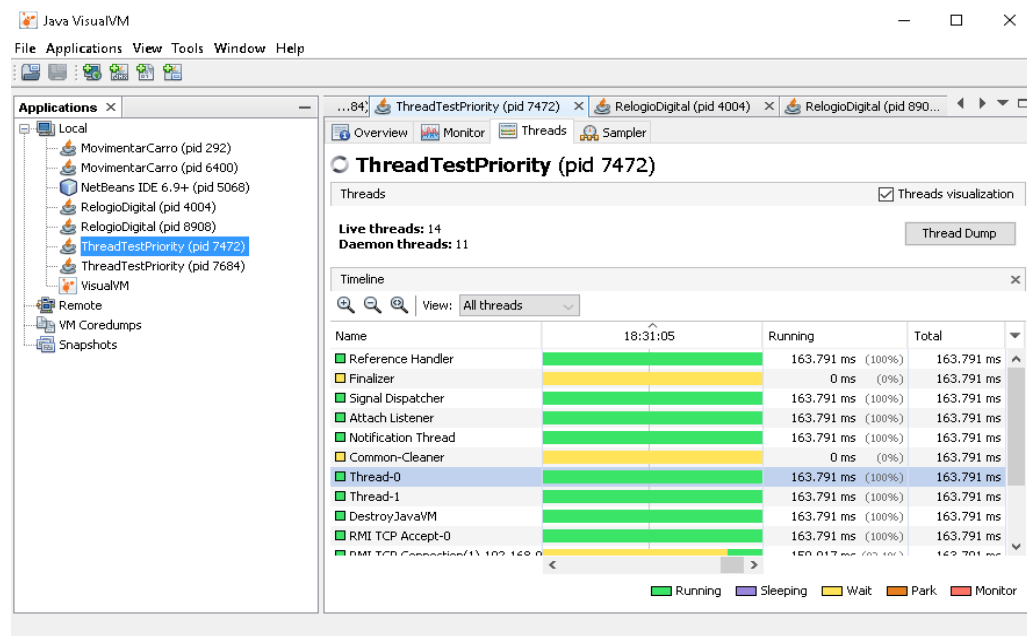
Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.2.1 Ciclo de vida (estados) de uma Thread

JDK tem a ferramenta muito útil `jvisualvm` que fornece uma interface visual para o programador ver informações profundas e detalhadas sobre aplicativos Java locais e remotos enquanto eles estão em execução (nota: um programa Java só termina quando as threads daemon terminam).

Ele ajuda os programadores e arquitetos a rastrear vazamentos de memória, analisar os dados heap, monitorar o Garbage Collector e o perfil da CPU. Também ajuda a melhorar o desempenho do aplicativo e garantir que o uso da memória seja otimizado. Com recursos como análise de thread e análise de head dump, é muito útil para resolver problemas de tempo de execução.





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Jan/2026

// Demonstrar que uma thread nao morre sem as instrucoes devidas

```
public class SeThreadMorreu1 {  
  
    public static void main( String[] args) {  
        new Thread() -> {  
            while( true ) {  
                System.out.println( "Thread executando..." );  
                try {  
                    Thread.sleep( 1000 );  
                }  
                catch ( InterruptedException ex ) {  
                    ex.printStackTrace();  
                }  
            }  
        }  
        ).start();  
        System.out.println( "Todas as instrucoes do main ja foram executadas" );  
    }  
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Jan/2026

// Demonstrar que uma thread morre com as instrucoes devidas

```
public class SeThreadMorreu2 {  
    public static void main( String[] args ) throws Exception {  
        Thread t = new Thread() -> {  
            while ( true ) {  
                System.out.println( "Thread executando..." );  
                try {  
                    Thread.sleep( 1000 );  
                }  
                catch ( InterruptedException ex ) {  
                    ex.printStackTrace();  
                }  
            }  
        });  
        t.setDaemon( true );  
        t.start();  
        System.out.println( "Todas as instrucoes do main ja foram executadas" );  
    }  
}
```



2.2.2 Prioridades e agendamento de uma Thread

- Cada *thread* Java tem uma prioridade associada que ajuda a determinar a ordem em que são agendadas para execução (recebendo um *quantum* de CPU para serem processadas);
- A prioridade varia num *range* de 1 (mínima) à 10 (máxima);
- Cada nova *thread* por *default* herda a prioridade da *thread* que a cria. Informalmente, as *threads* de prioridade mais alta são mais importantes para um programa e devem ser alocadas em tempo de processador antes das *threads* de prioridade mais baixa.
- As threads de igual prioridade são executadas num sistema *round-robin*.

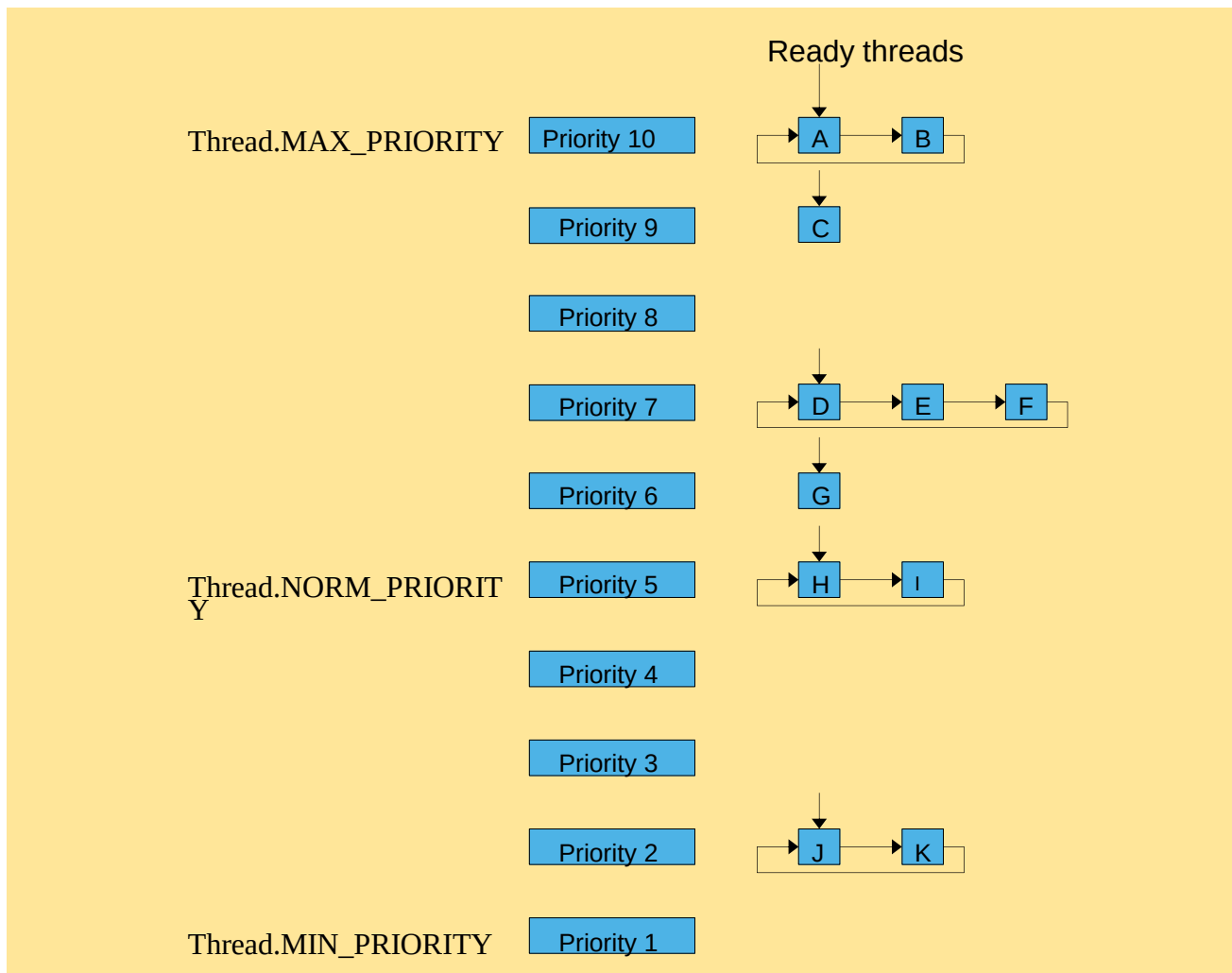


Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.2.2 Prioridades e agendamento de uma Thread





Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim

// Data: Jan/2026

// Demonstrar o uso de prioridades numa thread

```
public class ThreadTestPriority {  
    public static void main( String[] args ){  
        Thread r = Thread.currentThread();  
        r.setPriority( Thread.MAX_PRIORITY );  
        NewThread highPrioThread = new NewThread( Thread.MAX_PRIORITY );  
        NewThread lowPrioThread = new NewThread( Thread.MIN_PRIORITY );  
        highPrioThread.start();  
        lowPrioThread.start();  
        try {  
            r.sleep( 5000 );  
        }  
        catch( InterruptedException ex ) {  
            ex.printStackTrace();  
        }  
        highPrioThread.interrupt();  
        lowPrioThread.interrupt();  
    }  
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
System.out.println( "qtas vezes a HIGH foi processada = " + highPrioThread.counter );
System.out.println( "qtas vezes a LOW foi processada = " + lowPrioThread.counter );
    }
}

class NewThread extends Thread{
    long counter = 0;

    NewThread( int p ) {
        setPriority( p );
    }

    public void run() {
        while( true ) {
            counter++;
        }
    }
}
```